

① User

- Register & Login
- Browse products
- search products
- add to cart
- modify cart
- check out
- payment
- tracking } courier tracking [p.o.s]
- order history

(Accelerated checkout)

- ↳ combine
add to cart
& checkout on a
single button

② Admin

- add/update/remove products
- manage inventory
- view customer orders.
- update order status.

Entities

- User
- Address
- Product
- Category
- Inventory
- cart
- Cart Item
- Order
- Order Item
- Payment

(DB Schema)

User

- id [long, UUID] → long/string
- name → string
- email → string
- contact → string
- password-hash → string
- is_active → boolean
- created_at → timestamp
- updated_at → timestamp

→ updated-at

Address

- id
- user-id [FK]
- address-line-1
- address-line-2
- city
- state
- pincode
- country
- address-type
- lat } optional
- long }

In SQL DB, the FK is stored on the many (M) side.

In POJOs, the FK will be represented as the list of entities on the one (1) side.

class User {

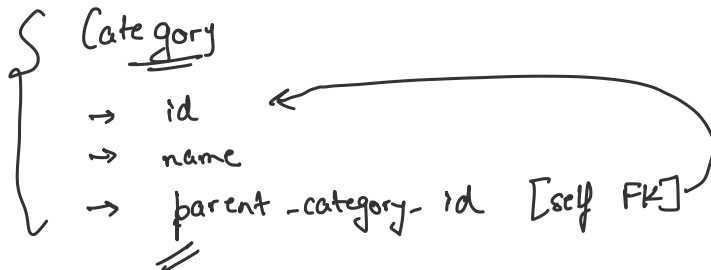
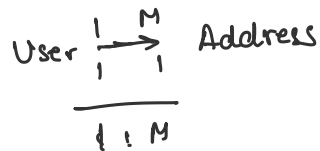
{ @One To Many
List <Address> addresses;

}

class Address {

{ @One To One
User user;

}



Product

- id
- name
- description
- sku [stock keeping unit]
- category-id
- price
- was-price / compare-at-price / retail-price
- is-active
- created-at
- updated-at

Inventory

→ id

- id
- product-id [FK]
- available-qty
- reserved-qty }
- warehouse-id }

Cart

- id
- user-id (FK)
- created-at
- updated-at

Cart Items

- id
- cart-id (FK)
- product-id (FK)
- quantity

Order

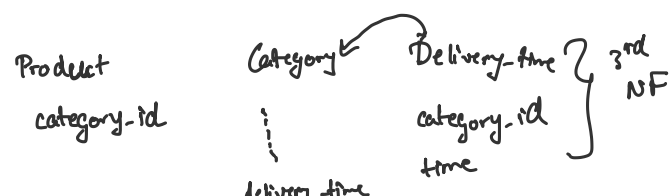
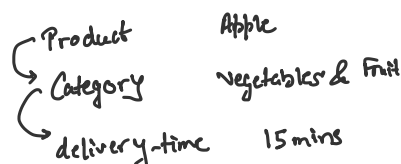
- id
- user-id (FK)
- address-id (FK)
- total-amount
- status (enum) - CREATED, IN-TRANSIT, DELIVERED, CANCELLED, FAILED, SHIPPED, PAID
- created-at
- updated-at

Order Items

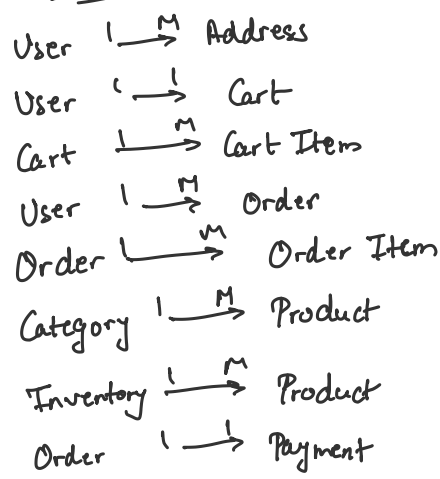
- id
- order-id (FK)
- product-id (FK)
- quantity
- price-at-purchase.

Payment

- id
- order-id
- amount
- status
- payment-method
- reference-number.



Relationships



Services

① Auth Service

register()

login()

validate() $\rightarrow$ validate if current session is valid

② Product Service

- $\rightarrow$ getProduct() $\rightarrow$ fetch single product (mostly by id)
- $\rightarrow$ createProduct()
- $\rightarrow$ updateProduct()
- $\rightarrow$ searchProduct() $\rightarrow$ fetch list of product based on condition
- $\rightarrow$ deleteProduct()

✓ ③ Cart Service

- $\rightarrow$ addToCart()
- $\rightarrow$ removeFromCart()
- $\rightarrow$ updateQuantity()
- $\rightarrow$ fetchCart()

④ Inventory Service

- $\rightarrow$ checkAvailability(product Id) $\rightarrow$ check stock levels.
- $\rightarrow$ reserveStock()
- $\rightarrow$ releaseStock()
- $\rightarrow$ deductStock()
- $\rightarrow$ updateStock()

✓ ⑤ Order Service

- $\rightarrow$ createOrder()
- $\rightarrow$ getOrder()

→ update Order Status()

⑥ Payment Service

- initiate Payment
- validate Payment ?
- fetch Payment Details()

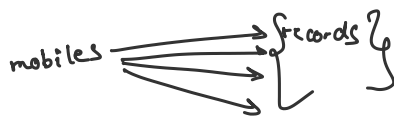
DB type

- SQL DBs for transactional data
- Key/Value based DB (DDB/Redis) for caching / locking.
- (Elastic Search for searching)

Elastic Search

maintains information as json documents

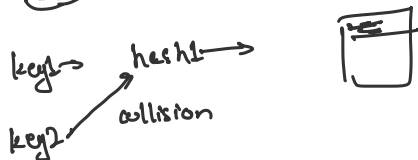
uses reverse indexing.



efficient for search

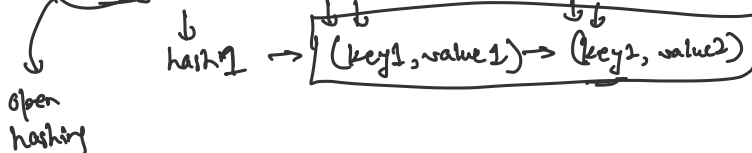
slow writes / updates.

Hash



prevent by using better hash functions.

Chaining



search by hash → O(1)

when collisions happens $\rightarrow$
 $\hookrightarrow$ search on hash chains
 $\hookrightarrow O(N)$

Java 8

$\hookrightarrow$ hash collisions $\rightarrow$ create chain

when chain size > 15 $] O(N)$

$\hookrightarrow$ convert to BST

$\hookrightarrow O(\log N)$